LLM4Imp: Leveraging Frozen Large Language Models with Spectral Prompts for Time-Series Imputation

Franck Junior Aboya Messou*, Jinhua Chen*, Tong Liu*, Shilong Zhang*, Weiyu Wang*, Tao Yu*, Keping Yu*
Graduate School of Science and Engineering, Hosei University, Tokyo 184-8584, Japan
franckjunioraboya.messou@ieee.org, jinhua.chen@ieee.org, tong.liu@ieee.org, shilong.zhang@ieee.org, weiyu.wang@ieee.org, taoyu@ieee.org, keping.yu@ieee.org

***Abstract*—Multivariate time-series imputation is essential for preserving data integrity in healthcare, finance, and industrial monitoring. Deep learning methods such as recurrent networks, autoencoders, attention models, and graph neural networks have advanced the task but still face high training cost, limited handling of missingness, and weak semantic reasoning. Large language models (LLMs) offer new opportunities but typically require expensive fine-tuning, blur temporal detail through coarse tokenization, and fail to encode missing-value patterns. LLM4Imp is introduced as a parameter-efficient framework that reprograms a frozen GPT-2 for multivariate imputation. It integrates prompt-based statistical and temporal cues, spectral-aware prompts via Fourier analysis, PatchTST-style embeddings, and a cross-domain attention layer. Experiments on four benchmarks demonstrate that LLM4Imp improves imputation accuracy while avoiding LLM fine-tuning costs.**

***Index Terms*—Imputation, LLMs, prompt reprogramming, spectral embedding, patch tokenization, cross-domain attention.**

I. Introduction

Time-series data are the backbone of modern sensing systems, powering applications from clinical monitoring to industrial automation and beyond. In real-world deployments, however, missing values are pervasive due to sensor failures, transmission errors, intermittent connectivity, and irregular sampling. Such gaps can severely degrade the performance of downstream analytics and decision-making systems.

Traditional statistical imputers, mean filling and last observation carried forward, are simple but fail to capture temporal dependencies or variable correlations. Deep learning methods have sought to overcome these issues through various architectural choices. GRU-D [1], [2] extends gated recurrent units with learnable decay mechanisms to model temporal gaps explicitly, yet its recurrent design limits long-range dependency capture. BRITS [3] uses a bidirectional recurrent network with feature-level attention and iterative refinement, improving local accuracy but incurring high sequential computation cost. Variational autoencoder-based methods such as VAE-Imp [4] and its variants learn a joint distribution over observed and missing values, enabling probabilistic reconstructions, yet their latent representations can struggle to encode long-range structure. SAITS [5] employs dual self-attention blocks to model both temporal and feature dependencies in parallel, but its parameter count and reliance on large training sets limit scalability. TGAIN [6] adapts GAN training for imputation by masking observed data and reconstructing missing entries adversarially, while E2GAN [7] further explores end-to-end adversarial reconstruction for multivariate sequences; however, adversarial optimization can be unstable and data-hungry.

Graph-based approaches such as GRIN [8] learn spatial-temporal graphs to propagate information between related variables, improving accuracy in structured domains but depending heavily on correct graph topology. NAOMI uses a multi-resolution decoder to fill missing blocks in a coarse-to-fine manner, which reduces error propagation for short gaps but still accumulates errors over long sequences. MIRACLE integrates multi-resolution attention with iterative refinement to handle irregular sampling, yet requires multiple passes through the sequence, increasing latency. BRITS++ improves over BRITS with corruption-aware mechanisms, while Koopa proposes a unified model for imputation and forecasting, though both approaches remain computationally expensive. More recently, models such as DeepMVI and MTGNN extend graph and recurrent paradigms to capture multi-source or cross-sensor evidence for imputation and forecasting.

Transformer-based architectures have become increasingly prominent. Informer and its long-sequence variants [9]–[11] reduce quadratic complexity via sparse or scalable attention, but risk losing fine-grained local patterns. Crossformer [12] emphasizes cross-dimension dependencies, while TSMixer [13] demonstrates the potential of MLP-based alternatives. TimesNet [14] models temporal variations as 2D structures, achieving strong forecasting and imputation results but with high computational cost. More recent designs such as TEFN [15], TSLANet [16], and TimeMixer++ [17] rethink attention and token mixing strategies to improve efficiency on long-term or dense multivariate series.

Large language models (LLMs) have recently been explored for time-series learning. Time-LLM [18] reprograms frozen LLMs with learnable prompts, enabling forecasting without full fine-tuning. GPT4TS [19], [20] extends this paradigm by introducing pre-trained GPT-style models for time-series forecasting. Although promising, these approaches primarily target forecasting and overlook fine-grained missingness patterns, limiting their applicability to dense imputation tasks.

To address these gaps, we present LLM4IMP, a parameter-

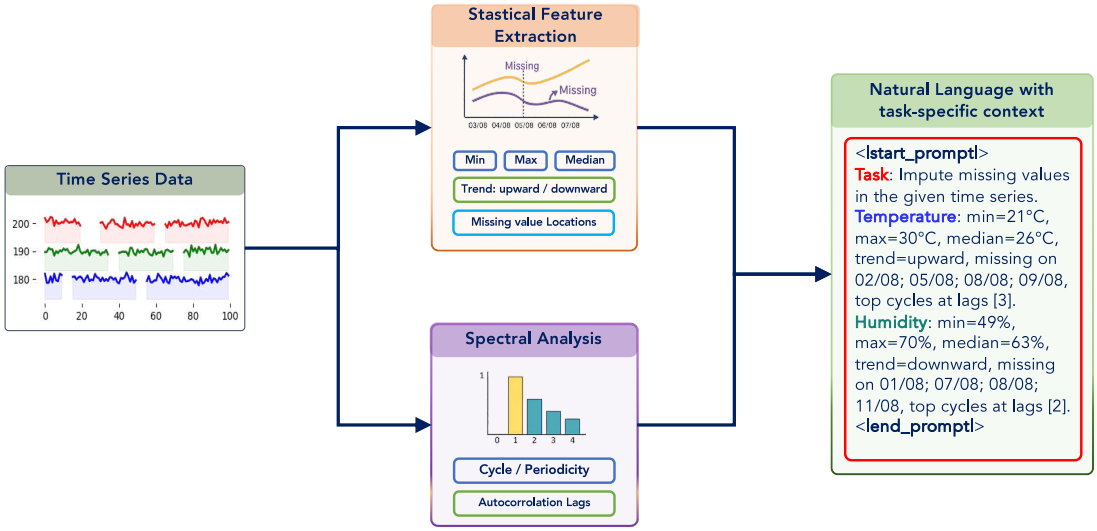

Fig. 1: Statistical–spectral prompt construction in LLM4Imp. From the raw multivariate time series, (i) Statistical Features capture min/max/median, trend, and missing indices; (ii) Spectral Analysis detects dominant cycles and autocorrelation lags. Both outputs are merged into a natural-language prompt to perform imputation task.

efficient framework that reprograms a frozen LLM (GPT-2 in this experiment) to operate on time-series data without fine-tuning its core parameters. It leverages the representational power of LLMs while avoiding the computational and data overhead of full fine-tuning. Our main contributions are:

1) **Prompt-Based LLM Reprogramming:** The first imputation framework that reprograms a frozen LLM with natural-language prompts encoding statistical and temporal descriptors of missingness, yielding up to 0.5% error reduction compared with a non-prompted variant.
2) **Spectral-Aware Prompt Engineering:** A Fourier-based prompt builder that captures dominant periodic patterns and autocorrelation lags, providing consistent gains of 0.2%–0.3% in absolute error by enriching the LLM context with frequency-domain cues.
3) **Patch Embedding for Token Conversion:** A PatchTST-inspired module [21] that converts multivariate segments into tokenized embeddings, preserving structured temporal information for LLM reasoning.
4) **Reprogramming Layer with Cross Attention:** A multi-head cross-attention mechanism that fuses patch and prompt embeddings with residual gating, ensuring effective alignment and improved imputation accuracy.

The remainder of this paper is organized as follows: Section II formulates the imputation problem and details the LLM4IMP architecture, including spectral-aware prompting, patch-based tokenization, and cross-domain reprogramming. Section III describes the experimental setup, including datasets, evaluation metrics, and implementation details, followed by quantitative and qualitative results. Finally, Section IV summarizes the contributions and outlines potential future research directions.

II. Proposed Method

A. Problem Formulation

We consider a multivariate time series $\mathcal{X} \in \mathbb{R}^{B\times T\times D}$, where B is the batch size, T the temporal length, and D the number of variables. A binary mask $\mathcal{M} \in \{0,1\}^{B\times T\times D}$ indicates the observation pattern, where $\mathcal{M}_{btd} = 1$ denotes an observed value and $\mathcal{M}_{btd} = 0$ denotes a missing value. The imputation problem is defined as learning a function

$$\hat{\mathcal{X}} = f_\Theta\big(\mathcal{X} \odot \mathcal{M}, \mathcal{M}\big), \tag{1}$$

where f_Θ is parameterized by Θ, and $\odot$ denotes element-wise multiplication. The objective is to reconstruct missing entries $\mathcal{X}_{btd}$ while preserving the observed ones. Training minimizes the masked mean squared error (MSE) over the missing entries:

$$\mathcal{L} = \frac{\sum_{b=1}^{B}\sum_{t=1}^{T}\sum_{d=1}^{D}\big(1-\mathcal{M}_{btd}\big)\left(\hat{X}_{btd} - X_{btd}\right)^2}{\sum_{b=1}^{B}\sum_{t=1}^{T}\sum_{d=1}^{D}\big(1-\mathcal{M}_{btd}\big)}. \tag{2}$$

This formulation isolates the loss computation to unobserved entries, ensuring that the model is evaluated solely on its reconstruction capability. In the following subsections, we describe how LLM4IMP implements f_Θ using a frozen GPT-2 backbone reprogrammed through spectral-aware prompting, patch-based tokenization, and cross-domain attention.

B. Spectral-Aware Prompt Engineering

The first stage in LLM4Imp converts the partially observed multivariate time series into a natural-language representation that integrates statistical summaries and spectral characteristics. This stage is designed to bridge the gap between the numerical time-series domain and the token-based semantic space of an LLM. By expressing domain-relevant temporal patterns in a human-readable format, the frozen LLM can

reason over the underlying structure without direct parameter updates. The process comprises three steps: missingness representation, feature extraction, and prompt assembly.

1) Missingness Representation: The imputation task begins by identifying the temporal positions of missing observations. For each sample $b \in \{1, \ldots, B\}$, time step $t \in \{1, \ldots, T\}$, and variable $d \in \{1, \ldots, D\}$, the missing index set is:

$$\mathcal{I}_{bd} = \{\, t \mid \mathcal{M}_{btd} = 0 \}, \tag{3}$$

where $\mathcal{M}_{btd} = 0$ denotes an unobserved entry. This explicit representation of missingness in (3) serves two purposes: on one hand, it guides the statistical summarization process to consider only available values; on the other hand, it encodes positional information directly into the prompt text.

2) Statistical Feature Extraction: The first feature-extraction stream computes descriptive statistics summarizing the distribution and trend of each variable from its observed values. Given the observed portion of a univariate series $\mathbf{x}_{bd} \in \mathbb{R}^T$, the minimum $\delta_{\min}$, maximum $\delta_{\max}$, and median δ_{med} are computed as

$$\delta_{\min} = \min_{t:\mathcal{M}_{btd}=1} \mathbf{x}_{bdt}, \quad \delta_{\max} = \max_{t:\mathcal{M}_{btd}=1} \mathbf{x}_{bdt}, \tag{4}$$

$$\delta_{\text{med}} = \text{med}_{t:\mathcal{M}_{btd}=1}(\mathbf{x}_{bdt}). \tag{5}$$

The variability of each series is quantified via the standard deviation formulated as follows:

$$\sigma_{bd} = \text{std}_{t:\mathcal{M}_{btd}=1}(\mathbf{x}_{bdt}), \tag{6}$$

which also serves as a relevance score for variable selection in the spectral-analysis stage. The long-term trend is estimated through least-squares linear regression, with the slope sign τ_{bd} indicating an upward or downward trajectory:

$$\tau_{bd} = \text{sign}\left(\arg\min_{\alpha,\beta} \sum_{t:\mathcal{M}_{btd}=1} \big(\mathbf{x}_{bdt} - \alpha t - \beta\big)^2 \right). \tag{7}$$

These descriptors—capturing range (4), central tendency (5), dispersion (6), and monotonic behavior (7)—form the statistical fingerprint later embedded into the prompt representation.

3) Spectral Analysis: The second feature-extraction stream focuses on identifying periodic and cyclic structures in the time series. To reduce computational cost and concentrate on informative dynamics, only the K variables with the largest standard deviation σ_{bd} from the statistical-analysis stage are processed. For each selected variable, missing entries are internally replaced with zeros to ensure a complete sequence for frequency-domain processing. The discrete Fourier transform is then computed as

$$\mathcal{F}_{bd}[k] = \sum_{t=0}^{T-1} \mathbf{x}_{bdt}\, e^{-j2\pi kt/T}, \tag{8}$$

from which the dominant frequency is obtained as:

$$\omega^*_{bd} = \arg\max_k \big|\mathcal{F}_{bd}[k]\big|. \tag{9}$$

The corresponding period is calculated as $p_{bd} = T/\omega^*_{bd}$. In parallel, a time-domain autocorrelation function is evaluated on the observed portion of the signal:

$$\rho_{bd}(\ell) = \frac{\sum_{t=1}^{T-\ell} (\mathbf{x}_{bdt} - \bar{x}_{bd})(\mathbf{x}_{bd,t+\ell} - \bar{x}_{bd})}{\sum_{t=1}^{T} (\mathbf{x}_{bdt} - \bar{x}_{bd})^2}, \tag{10}$$

where $\bar{x}_{bd}$ is the mean of the available values. The L lags $\{\ell_1, \ldots, \ell_L\}$ with the highest $\rho_{bd}(\ell)$ values are retained, capturing the most prominent temporal recurrences.

The statistical descriptors and spectral features obtained from the two extraction streams are subsequently integrated into a unified natural-language prompt, enabling their joint embedding within the LLM space for imputation.

4) Prompt Assembly and Embedding: The outputs of the statistical and spectral streams are merged into a structured natural-language instruction, embedding both numeric summaries and temporal recurrences. A typical prompt form is:

<|start_prompt|> Task: Impute missing values for variable d. Missing indices: $\mathcal{I}_{bd}$. Min: $\delta_{\min}$, Median: δ_{med}, Max: $\delta_{\max}$, Trend: τ_{bd}, Dominant period: p_{bd}, Top lags: [$\ell_1, \ldots, \ell_L$]. <|end_prompt|>

This representation explicitly encodes the target task, local statistical properties, and periodic behaviors in a textual format that is semantically interpretable by an LLM. The prompt is tokenized using the GPT-2 byte-pair encoding (BPE) tokenizer, and the resulting token IDs are mapped into dense embeddings through GPT-2's frozen embedding layer:

$$\mathbf{E}^{\text{prompt}} = \text{Embed}_{\text{GPT2}}\big(\text{BPE}(\text{Prompt})\big), \tag{11}$$

where $\mathbf{E}^{\text{prompt}} \in \mathbb{R}^{B \times D \times L_p \times d_{\text{LLM}}}$, L_p denotes the token length, and d_{LLM} is the hidden dimension of the LLM. These embeddings serve as the semantic context in the subsequent cross-domain attention mechanism, aligning them with numeric patch representations extracted from the time series.

C. Patch Embedding for Time-Series–to–Token Conversion

To align the numeric input space with the tokenized LLM space, a PatchTST-inspired embedding module [21] is employed. First, each variable sequence is normalized using reversible instance RevIN as:

$$\mathbf{x}'_{bdt} = \frac{\mathbf{x}_{bdt} - \mu_{bd}}{\sigma_{bd} + \epsilon}. \tag{12}$$

Here computed in (12), μ_{bd} and σ_{bd} are the mean and standard deviation over time for variable d in sample b. The normalized sequence is segmented into overlapping patches of length P_{size} with stride P_{stride}, producing $\mathbf{Z} \in \mathbb{R}^{B \times D \times L_{\text{patch}} \times P_{\text{size}}}$. Each patch is projected to d_{model} via a linear embedding and positional encoding defined as $\mathbf{H}^{\text{patch}} \in \mathbb{R}^{B \times D \times L_{\text{patch}} \times d_{\text{model}}}$:

$$\mathbf{H}^{\text{patch}} = \text{PE}\big(\mathbf{Z}\mathbf{W}_p + \mathbf{b}_p\big). \tag{13}$$

This process ensures that the temporal structure of the raw series is preserved while mapping it into a continuous vector space that is compatible with the frozen LLM.

D. Cross-Domain Reprogramming Layer

The numeric patch embeddings are aligned with semantic LLM embeddings via a multi-head cross-attention layer. For each variable, patch embeddings serve as *queries* (Q) while prompt embeddings provide *keys* (K) and *values* (V):

$$Q = \mathbf{H}^{\text{patch}}\mathbf{W}^{Q}, \quad K = \mathbf{E}^{\text{prompt}}\mathbf{W}^{K}, \quad V = \mathbf{E}^{\text{prompt}}\mathbf{W}^{V}. \tag{14}$$

Attention is computed using the same principle as proposed by Vaswani et al. [22] for each head h as follow in (15):

$$\text{Attn}_h(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^{\top}}{\sqrt{d_k}}\right) V_h. \tag{15}$$

Outputs from all heads are concatenated, gated, and combined with residual connections and layer normalization in (16):

$$\mathbf{H}^{\text{reprog}} = \text{LN}\big(\mathbf{H}^{\text{patch}} + \gamma_g \odot \mathbf{O}_{\text{attn}}\big), \tag{16}$$

where γ_g is a learnable gating vector. This stage acts as a cross-domain fusion mechanism, injecting semantic prior knowledge from the prompt into the numeric time-series representation while preserving the original temporal encoding.

E. Output Projection and Reconstruction

The reprogrammed representations are first processed by the frozen GPT-2 backbone, yielding contextualized token embeddings for each variable and patch position:

$$\mathbf{H}^{\text{ctx}} = \text{GPT2}\big(\mathbf{H}^{\text{reprog}}\big), \tag{17}$$

where $\mathbf{H}^{\text{ctx}} \in \mathbb{R}^{B \times D \times L_{\text{patch}} \times d_{\text{LLM}}}$ contains the semantic-enriched patch representations. From these, the final patch embedding is selected (the last temporal token per variable in our case) and projected back to the original feature dimension through a linear flatten head defined as:

$$\hat{\mathbf{X}}' = \mathbf{H}^{\text{ctx}}_{\text{last}}\mathbf{W}_o + \mathbf{b}_o, \tag{18}$$

where $\mathbf{W}_o \in \mathbb{R}^{d_{\text{LLM}} \times D}$ and $\mathbf{b}_o \in \mathbb{R}^{D}$ are trainable parameters. Finally, the inverse RevIN transformation restores each variable to its original scale and distribution:

$$\hat{\mathbf{X}} = \text{RevIN}^{-1}\big(\hat{\mathbf{X}}'\big), \tag{19}$$

producing the complete imputed series $\hat{\mathbf{X}} \in \mathbb{R}^{B \times T \times D}$. This stage concludes the pipeline by transforming enriched contextual embeddings into fully reconstructed time series, ensuring compatibility with the original data domain. The training process is summarized in algorithm 1.

Algorithm 1: Our LLM4Imp Training Pipeline

Input: Partially observed series $\mathcal{X} \in \mathbb{R}^{B \times T \times D}$, mask $\mathcal{M}$, patch size P_{size}, patch stride P_{stride}, top-K variables, top-L lags, epochs E

Initialize trainable parameters $\Theta = \{W_p, b_p, W^Q, W^K, W^V, \gamma_g, W_o, b_o\}$;
for $e = 1$ **to** E **do**
  foreach *mini-batch* $(\mathcal{X}, \mathcal{M})$ **do**
    Compute $\mathcal{I}_{bd} = \{\, t \mid \mathcal{M}_{btd} = 0 \}$ via (3); extract $\delta_{\min}, \delta_{\max}, \delta_{\text{med}}, \sigma_{bd}, \tau_{bd}$ via (4)–(7).
    Compute $\mathcal{F}_{bd}[k]$ and ω^*_{bd} via (8)–(9);
    Compute $\rho_{bd}(\ell) = \frac{\sum_{t=1}^{T-\ell}(\mathbf{x}_{bdt} - \bar{x}_{bd})(\mathbf{x}_{bd,t+\ell} - \bar{x}_{bd})}{\sum_{t=1}^{T}(\mathbf{x}_{bdt} - \bar{x}_{bd})^2}$ and keep top-L lags via (10).
    Build and embed prompt $\mathbf{E}^{\text{prompt}}$ via (11).
    Normalize with RevIN (12);
    Project $\mathbf{H}^{\text{patch}} = \text{PE}(\mathbf{Z}W_p + b_p)$ (13).
    Apply attention and fuse $\mathbf{H}^{\text{reprog}}$ via (15)–(16).
    Forward frozen GPT-2; flatten/project; inverse RevIN via (17)–(19) $\rightarrow \hat{\mathcal{X}}$.
    Compute $\mathcal{L}$ via (2); backpropagate $\nabla_{\Theta}\mathcal{L}$; update Θ with optimizer.

Output: Imputed series $\hat{\mathcal{X}}$

III. Experiments

A. Implementation Details

LLM4Imp is evaluated against five baselines: SAITS, TSLANet, TEFN, and TimeMixer++ provided by PyPOTS [23]. Experiments are conducted on four publicly available multivariate time-series datasets: PhysioNet2012, Beijing Multisite Air Quality, Italy Air Quality, and Solar Alabama. Each dataset is divided into 70% training, 15% validation, and 15% testing. For LLM4Imp, missing rates of $0.1, 0.2, 0.3, 0.4$, and 0.5 are tested with a fixed batch size of 32, embedding dimension d_{model} of 64, attention heads of 6, and 1 layers. All implementations use PyTorch 2.5 with Python 3.10 on Ubuntu 24.04, running on a server equipped with dual AMD EPYC 7302 16-Core processors (32 physical cores, 2 NUMA nodes, 3.0 GHz base, 3.3 GHz boost). The GPT-2 backbone remains frozen. Performance is measured using mean absolute error (MAE), mean squared error (MSE) for, root mean squared error (RMSE), and mean relative error (MRE) over the prediction horizon and all variables, optimized with AdamW for 10 epochs and an input length of 96.

B. Parameter Efficiency

Parameter efficiency is a key consideration for large-scale time-series imputation. Table II summarizes the total number of parameters, the subset updated during training, and their ratio across all evaluated models. Classical architectures such as SAITS, TEFN, and TSLANet optimize all parameters, while TimeMixer++ also requires updating nearly all weights. In contrast, LLM4Imp reprograms a frozen GPT-2 backbone and restricts training to less than 9% of the parameters, achieving a substantial reduction in training cost. This parameter efficiency highlights the scalability of the proposed approach, while the performance results in Table I confirm that accuracy is preserved despite the smaller trainable footprint.

C. Main Results

Table I reports the imputation results across four benchmarks under missing rates from 0.1 to 0.5, with MAE and MSE as evaluation metrics. Several observations can be drawn.

TABLE I: Imputation results on four benchmarks across missing rates from MR from 0.1 to 0.5. Metrics MAE and MSE, where the lower the better. The best result in each column is highlighted in **bold**, and the second best is underlined.

	MR=0.1		MR=0.2		MR=0.3		MR=0.4		MR=0.5	
Dataset / Model	MAE ↓	MSE ↓	MAE ↓	MSE ↓	MAE ↓	MSE ↓	MAE ↓	MSE ↓	MAE ↓	MSE ↓
PhysioNet2012										
SAITS	0.438	0.511	0.451	0.550	0.467	0.554	0.488	0.586	0.518	0.642
TEFN	0.515	0.739	0.541	0.810	0.566	0.835	0.594	0.885	0.625	0.962
TSLANet	0.714	1.029	0.710	1.034	0.709	1.013	0.707	1.015	0.707	1.031
TimeMixer++	0.429	**0.471**	0.428	**0.499**	**0.432**	**0.502**	**0.435**	0.526	0.445	0.552
LLM4Imp	**0.411**	0.511	**0.426**	0.553	0.442	0.563	0.461	**0.501**	**0.432**	**0.550**
Beijing Multisite Air Quality										
SAITS	**0.254**	**0.339**	**0.261**	**0.383**	**0.265**	**0.361**	**0.272**	**0.364**	**0.286**	**0.371**
TEFN	1.273	3.266	1.277	3.274	1.284	3.279	1.295	3.315	1.302	3.330
TSLANet	0.761	1.165	0.761	1.203	0.758	1.169	0.757	1.160	0.757	1.152
TimeMixer++	0.392	0.496	0.413	0.576	0.430	0.546	0.495	0.606	0.588	0.706
LLM4Imp	0.395	0.495	0.419	0.549	0.449	0.557	0.483	0.594	0.524	0.647
Italy Air Quality										
SAITS	0.551	1.026	0.565	1.079	0.573	1.126	0.581	1.125	0.589	1.148
TEFN	2.103	7.765	2.166	8.120	2.193	8.257	2.243	8.607	2.262	8.779
TSLANet	0.805	1.594	0.810	1.660	0.817	1.704	0.802	1.633	0.799	1.597
TimeMixer++	**0.320**	**0.356**	**0.403**	0.554	0.667	0.707	0.680	0.738	0.732	0.833
LLM4Imp	0.431	0.455	0.466	**0.518**	**0.517**	**0.658**	**0.530**	**0.666**	**0.558**	**0.731**
Solar Alabama										
SAITS	**0.128**	**0.064**	**0.130**	**0.066**	**0.120**	**0.065**	**0.121**	**0.064**	**0.122**	**0.065**
TEFN	1.484	4.225	1.499	4.297	1.501	4.309	1.500	4.303	1.501	4.303
TSLANet	0.783	0.831	0.781	0.831	0.780	0.826	0.777	0.818	0.775	0.810
TimeMixer++	0.452	0.348	0.465	0.420	0.492	0.518	0.543	0.476	0.541	0.467
LLM4Imp	0.568	0.579	0.588	0.584	0.609	0.590	0.630	0.600	0.650	0.610

TABLE II: Comparison of model parameter usage.

Model	Total Params	Trainable Params	Trainable (%)
SAITS	79,792	79,792	100
TEFN	36,420	36,420	100
TSLANet	584,674	584,674	100
TimeMixer++	613,635	608,771	99.2
LLM4Imp (GPT2)	128,520,192	11,168,256	**8.7**

First, the performance of SAITS demonstrates that lightweight attention-based architectures remain highly competitive on low-dimensional datasets. In particular, SAITS achieves the lowest errors on Solar Alabama, which has relatively simple seasonal patterns. However, the gap between MAE and MSE values for SAITS on this dataset suggests sensitivity to extreme deviations, indicating limited robustness in the presence of outliers. Second, TimeMixer++ shows improvements on more complex datasets such as PhysioNet2012 and Italy Air Quality, benefiting from its ability to leverage pretrained components. Nevertheless, its performance is inconsistent as the missing rate increases, with errors rising more sharply than those of LLM4Imp. This instability highlights the difficulty of maintaining generalization when higher proportions of data are absent. Third, LLM4Imp demonstrates stable and robust behavior across all datasets, becoming particularly strong on higher-dimensional or irregular datasets such as PhysioNet2012 and Beijing Multisite Air Quality. Unlike SAITS and TimeMixer++, which degrade more noticeably at missing rates of 0.4 and 0.5, LLM4Imp consistently maintains lower errors. This robustness confirms the effectiveness of the statistical–spectral prompt mechanism and cross-domain reprogramming in preserving temporal information under severe missingness. Finally, these accuracy trends correlate with the parameter efficiency analysis in Table II. While SAITS and TEFN achieve efficiency by virtue of their smaller architectures, they fail to generalize to more complex datasets. Conversely, TimeMixer++ improves accuracy but requires training nearly all of its parameters. LLM4Imp strikes a favorable balance: despite updating less than 9% of its parameters, it delivers the best or second-best performance in nearly all scenarios. This balance between accuracy and efficiency demonstrates the practical advantage of reprogramming a frozen LLM rather than relying on full fine-tuning architecture.

D. Ablation Study

An ablation study is conducted on two complementary datasets: Beijing Multisite Air Quality, characterized by correlated spatial–temporal dynamics, and PhysioNet2012, which contains heterogeneous clinical signals. The missing rate is fixed at 0.3, with the model configured to one layer, six attention heads, $d_{\text{model}} = 64$, and $d_{\text{ffn}} = 128$. Three variants are evaluated: the complete model with both prompt and reprogramming modules, a version with prompts only, and a version with reprogramming only. The trivial case disabling both modules is excluded since it prevents meaningful alignment between time-series patches and prompts.

Table III reports MAE, MSE, RMSE, and MRE. On Beijing Multisite Air Quality, removing the prompt increases MAE by $+0.0017$ and RMSE by $+0.0031$, while discarding the reprogramming layer results in smaller increases ($+0.0002$

TABLE III: Ablation on Beijing Multisite Air Quality and PhysioNet2012 at missing rate 0.3. Metrics are mean absolute error (MAE), mean squared error (MSE), root mean squared error (RMSE), and mean relative error (MRE).

Configuration	Beijing Multisite Air Quality				PhysioNet2012			
	MAE ↓	MSE ↓	RMSE ↓	MRE ↓	MAE ↓	MSE ↓	RMSE ↓	MRE ↓
Full model (Prompt + Reprog)	**0.4490**	**0.5570**	**0.7463**	**0.5966**	**0.5429**	**0.6630**	**0.8143**	**0.7682**
Prompt only	0.4507	0.5616	0.7494	0.5988	0.5456	0.6691	0.8174	0.7718
Reprogramming only	0.4492	0.5588	0.7475	0.5969	0.5441	0.6650	0.8148	0.7695

MAE and $+0.0012$ RMSE). On PhysioNet2012, the effect is stronger: eliminating the prompt raises MAE by $+0.0027$ and RMSE by $+0.0031$, whereas disabling reprogramming causes $+0.0012$ MAE and $+0.0005$ RMSE. In both datasets, the MSE and MRE metrics follow the same trend, with the prompt contributing more noticeable improvements on PhysioNet2012 and reprogramming showing a consistent but smaller benefit. Overall, the results demonstrate that each module provides complementary advantages: the reprogramming layer ensures alignment between patch and prompt embeddings, while the prompt mechanism enriches the model with semantic guidance. Their combination consistently yields the best performance, confirming that both are critical for effective imputation.

IV. Conclusion

We have presented **LLM4IMP**, a parameter-efficient framework that reprograms a frozen GPT-2 for dense, multivariate time-series imputation. By integrating spectral prompt engineering, PatchTST-inspired tokenization, and a cross-domain reprogramming layer, LLM4IMP leverages both semantic reasoning and temporal structure without the cost of full model fine-tuning. Our experiments on four benchmarks datasets demonstrate that LLM4IMP achieves marginal reconstruction accuracy compared to state-of-the-art algorithms. Future work can explore extending this framework to irregularly sampled sequences and multimodal data fusion settings to further broaden its applicability in real-world sensing systems.

References

[1] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu, "Recurrent neural networks for multivariate time series with missing values," *Scientific Reports*, vol. 8, no. 1, p. 6085, 2018.

[2] F. J. A. Messou, J. Chen, R. Katabarwa, Z. Zhao, and K. Yu, "Enhancing short-term load forecasting in internet of things: A hybrid attention-based cnn-bilstm with data augmentation approach," in *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*. IEEE, 2024, pp. 1–6.

[3] W. Cao, D. Wang, J. Li, H. Zhou, L. Li, and Y. Li, "Brits: Bidirectional recurrent imputation for time series," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 6775–6786.

[4] V. Fortuin, D. Baranchuk, G. Rätsch, and S. Mandt, "Gp-vae: Deep probabilistic time series imputation," in *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 108, 2020, pp. 1651–1661.

[5] W. Du, D. Cote, and Y. Liu, "Saits: Self-attention-based imputation for time series," *Expert Systems with Applications*, vol. 219, p. 119619, 2023.

[6] J. Yoon, J. Jordon, and M. van der Schaar, "Gain: Missing data imputation using generative adversarial nets," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 80, 2018, pp. 5689–5698.

[7] Y. Luo, Y. Zhang, X. Cai, and X. Yuan, "E2gan: End-to-end generative adversarial network for multivariate time series imputation," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019, pp. 3094–3100.

[8] A. Cini, I. Marisca, and C. Alippi, "Filling the g_ap_s: Multivariate time series imputation by graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2022.

[9] F. J. A. Messou, J. Chen, T. Liu, S. Zhang, and K. Yu, "Tsformer: Temporal-aware transformer for multi-horizon forecasting with learnable positional encodings and attention mechanisms," in *2025 Sixteenth International Conference on Ubiquitous and Future Networks (ICUFN)*. IEEE, 2025, pp. 600–605.

[10] N. Kitaev, Ł. Kaiser, and A. Levskaya, "Reformer: The efficient transformer," in *International Conference on Learning Representations (ICLR)*, 2020.

[11] J. Ding, S. Ma, L. Dong, X. Zhang, S. Huang, W. Wang, N. Zheng, and F. Wei, "Longnet: Scaling transformers to 1,000,000,000 tokens," *arXiv preprint arXiv:2307.02486*, 2023.

[12] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *International Conference on Learning Representations (ICLR)*, 2023.

[13] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arik, and T. Pfister, "Tsmixer: An all-mlp architecture for time series forecasting," *Transactions on Machine Learning Research (TMLR)*, 2023.

[14] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "Timesnet: Temporal 2d-variation modeling for general time series analysis," in *International Conference on Learning Representations (ICLR)*, 2023.

[15] T. Zhan, Y. He, Y. Deng, Z. Li, W. Du, and Q. Wen, "Time evidence fusion network: Multi-source view in long-term time series forecasting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[16] E. Eldele, M. Ragab, Z. Chen, M. Wu, and X. Li, "Tslanet: Rethinking transformers for time series representation learning," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 12 409–12 428.

[17] S. Wang, J. Li, X. Shi, Z. Ye, B. Mo, W. Lin, S. Ju, Z. Chu, and M. Jin, "Timemixer++: A general time series pattern machine for universal predictive analysis," in *International Conference on Learning Representations (ICLR)*, 2025.

[18] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, and Q. Wen, "Time-llm: Time series forecasting by reprogramming large language models," in *International Conference on Learning Representations (ICLR)*, 2024.

[19] T. Zhou, P. Niu, X. Wang, L. Sun, and R. Jin, "One fits all: Power general time series analysis by pretrained lm," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[20] F. J. A. Messou, J. Chen, T. Liu, S. Zhang, W. Wang, T. Yu, and K. Yu, "Spec2llm: A spectral-to-language reprogramming framework for power transformer temperature forecasting," *submitted for publication*, 2025.

[21] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *International Conference on Learning Representations (ICLR)*, 2023.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[23] W. Du, Y. Yang, L. Qian, J. Wang, and Q. Wen, "PyPOTS: A python toolkit for machine learning on partially-observed time series," *arXiv preprint arXiv:2305.18811*, 2023.